DAY 24

Make consumer processing safe to repeat without corrupting state.

IDEMPOTENT CONSUMER, DEDUPLICATION, AND DELIVERY GUARANTEES

Process the same message multiple times, produce one effective business outcome.

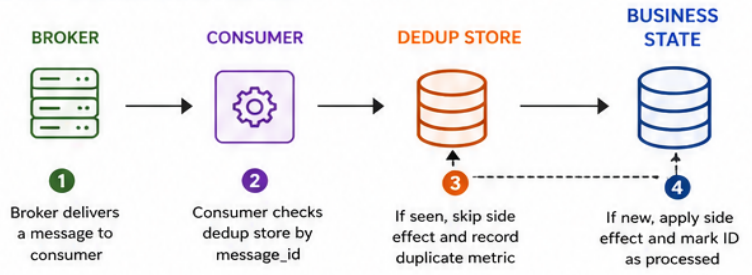
THE GOAL

Handle duplicates safely in at-least-once delivery systems and prevent side effects from happening twice.

WHY DUPLICATES HAPPEN

- Network failures and ack timeouts can trigger message redelivery.
- Producers and brokers may retry the same logical message.
- At-least-once delivery prioritizes durability over uniqueness.
- Duplicates are normal. Design consumers to be safe, not perfect.

HOW IT WORKS (HIGH LEVEL)



IDEMPOTENT CONSUMER PRINCIPLE

Processing the same message multiple times should produce one effective business outcome.

- Store a stable dedup key (message_id or business operation id).
- Check before side effects. Skip if already processed.
- Key choice should represent business uniqueness, not transport metadata.
- Idempotency makes retries safe. State stays correct.

DEDUPLICATION DESIGN

- Dedup store can be in-memory, cache, or durable DB table.
- Retention / TTL must cover expected replay and retry windows.
- Choose keys that map to business uniqueness (e.g., order_id, payment_id).

DEDUP STORE EXAMPLE

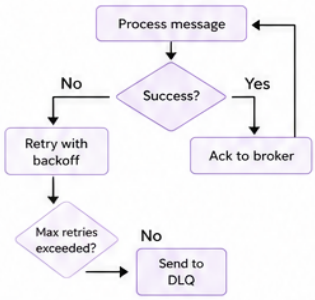
message_id	processed_at
msg_1001	2025-05-14 10:00:01
msg_1002	2025-05-14 10:00:03
msg_1003	2025-05-14 10:00:05
...	...

Use TTL or retention policy to clean up old keys safely.

FAILURE & RETRY HANDLING

- If side effect succeeds but ack fails, duplicate arrives later.
- Idempotency prevents charging, emailing, or updating inventory twice.
- Poison messages should be routed to dead-letter handling after bounded retries.

RETRY FLOW



TRADE-OFFS

- Strong dedup improves correctness but adds storage and coordination overhead.
- Very short TTL can reintroduce duplicates after expiry.
- Exactly-once is usually end-to-end expensive; idempotency is the practical baseline.

AT-LEAST-ONCE DELIVERY BEHAVIOR

- Message may be delivered 0, 1, or multiple times.
- Order may not be guaranteed across retries.
- Your consumer must be prepared for duplicates.

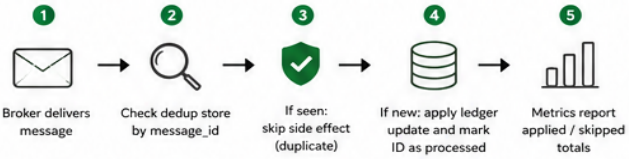
Same logical message delivered more than once

DAY-24 SAMPLE: IDEMPOTENT PAYMENT EVENT CONSUMER

COMPONENTS

- **Message** — Immutable event envelope (message_id, amount, etc.)
- **DedupStore** — Remembers processed message IDs
- **BalanceLedger** — Side-effect target: account balances
- **PaymentConsumer** — Idempotent handler and metrics

FLOW



METRICS EXAMPLE

Applied Messages (unique)	Skipped Duplicates	Duplicate Rate
125	42	25.15% (42 / 167)

NEXT IMPROVEMENTS

- Add TTL-based dedup retention policy
- Persist dedup keys in durable storage
- Add retry backoff and DLQ simulation
- Add per-partition ordering and keying strategy
- Add tracing and audit logs for replay investigations

> Idempotency is not optional in at-least-once systems.
> Design for duplicates. Protect correctness.

SAFE TO REPEAT.
CORRECT BY DESIGN.
TRUST IN EVERY RETRY.

20 DAYS OF SYSTEM DESIGN CHALLENGE

Day 24: Idempotent Consumer, Deduplication, and Delivery Guarantees

Day 24 focuses on how consumers process messages safely when brokers deliver duplicates due to retries and at-least-once semantics.

It covers:

- at-least-once delivery behavior
- idempotent consumer design
- deduplication key strategy
- side-effect safety under retries
- poison message and retry trade-offs
- operational observability for duplicate processing

In short, Day 24 is about making consumer processing safe to repeat without corrupting state.

Core ideas

Why duplicates happen

- Network failures and ack timeouts can trigger message redelivery.
- Producers and brokers may retry the same logical message.
- At-least-once delivery prioritizes durability over uniqueness.

Idempotent consumer principle

- Processing the same message multiple times should produce one effective business outcome.
- Consumer stores a stable dedup key (for example, message_id or business operation id).
- Already-seen keys are skipped before side effects run.

Deduplication design

- Dedup store can be in-memory, cache, or durable database table.
- Retention/TTL must cover expected replay and retry windows.
- Key choice should map to business uniqueness, not transient transport metadata.

Failure handling

- If side effect succeeds but ack fails, duplicate arrives later.
- Idempotency prevents charging, emailing, or updating inventory twice.
- Poison messages should be routed to dead-letter handling after bounded retries.

Trade-offs

- Strong dedup improves correctness but adds storage and coordination overhead.

- Very short TTL can reintroduce duplicates after expiry.
- Exactly-once is usually end-to-end expensive; idempotency is the practical baseline.

Day-24 sample: Idempotent Payment Event Consumer

This repository includes a small idempotent consumer sample in both Python and Java.

Functional requirements

- Consume payment events with message_id and amount
- Apply account balance side effect once per unique message
- Skip duplicate deliveries safely
- Track processing audit counters (applied vs skipped)
- Demonstrate duplicate-heavy input stream handling

High-level components

- Message: immutable event envelope
- DedupStore: remembers processed message IDs
- BalanceLedger: side-effect target state
- PaymentConsumer: idempotent message handler and metrics

Data flow

1. Broker delivers a message to consumer
2. Consumer checks dedup store by message_id
3. If seen, consumer skips side effect and records duplicate metric
4. If new, consumer applies ledger update and marks ID as processed
5. Metrics report applied/skipped totals for observability

Project structure

```
Day-24/
  README.md
  idempotent-consumer-sample/
    python/
      idempotent_consumer.py
      demo.py
    java/
      IdempotentConsumerSystem.java
      Main.java
```

Run the sample

Python

```
cd idempotent-consumer-sample\python
python demo.py
```

Java

```
cd idempotent-consumer-sample\java
javac Main.java IdempotentConsumerSystem.java
java Main
```

Interview takeaways

- At-least-once delivery requires idempotent consumers to preserve correctness
- Dedup keys should represent business uniqueness and survive retries
- Consumer metrics should expose duplicate rate and processing outcomes
- Dead-letter and retry strategy must be explicit in design discussions
- Idempotency is often more practical than strict exactly-once guarantees

Next improvements

- Add TTL-based dedup retention policy
- Persist dedup keys in durable storage
- Add retry backoff and DLQ simulation
- Add per-partition ordering behavior and keying strategy
- Add tracing and audit logs for replay investigations

Code Appendix

Python and Java source used in Day 24 sample.

idempotent-consumer-sample\python\idempotent_consumer.py

```
from __future__ import annotations

from dataclasses import dataclass


@dataclass(frozen=True)
class Message:
    message_id: str
    account_id: str
    amount_cents: int


class DedupStore:
    def __init__(self) -> None:
        self._seen: set[str] = set()

    def has_seen(self, message_id: str) -> bool:
        return message_id in self._seen

    def mark_seen(self, message_id: str) -> None:
        self._seen.add(message_id)


class BalanceLedger:
    def __init__(self) -> None:
        self._balances: dict[str, int] = {}

    def apply(self, account_id: str, amount_cents: int) -> None:
        self._balances[account_id] = self._balances.get(account_id, 0) + amount_cents

    def get_balance(self, account_id: str) -> int:
        return self._balances.get(account_id, 0)


class PaymentConsumer:
    def __init__(self, dedup_store: DedupStore, ledger: BalanceLedger) -> None:
        self._dedup = dedup_store
        self._ledger = ledger
        self.applied_count = 0
        self.duplicate_count = 0

    def handle(self, message: Message) -> str:
        if self._dedup.has_seen(message.message_id):
            self.duplicate_count += 1
            return f"skipped duplicate {message.message_id}"

        self._ledger.apply(message.account_id, message.amount_cents)
        self._dedup.mark_seen(message.message_id)
        self.applied_count += 1
        return f"applied {message.message_id} amount={message.amount_cents}"

    def balance(self, account_id: str) -> int:
        return self._ledger.get_balance(account_id)
```

idempotent-consumer-sample\python\demo.py

```
from idempotent_consumer import BalanceLedger, DedupStore, Message, PaymentConsumer


def main() -> None:
    dedup = DedupStore()
    ledger = BalanceLedger()
    consumer = PaymentConsumer(dedup, ledger)
```

```
    messages = [
        Message("msg-1", "acct-1", 1500),
        Message("msg-2", "acct-1", 2500),
        Message("msg-1", "acct-1", 1500),  # duplicate
        Message("msg-3", "acct-1", -700),
        Message("msg-2", "acct-1", 2500),  # duplicate
        Message("msg-4", "acct-2", 900),
    ]

    for message in messages:
        print(consumer.handle(message))

    print("acct-1 balance:", consumer.balance("acct-1"))
    print("acct-2 balance:", consumer.balance("acct-2"))
    print("applied_count:", consumer.applied_count)
    print("duplicate_count:", consumer.duplicate_count)


if __name__ == "__main__":
    main()
```

idempotent-consumer-sample\java\IdempotentConsumerSystem.java

```
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

public class IdempotentConsumerSystem {
    public static final class Message {
        private final String messageId;
        private final String accountId;
        private final int amountCents;

        public Message(String messageId, String accountId, int amountCents) {
            this.messageId = messageId;
            this.accountId = accountId;
            this.amountCents = amountCents;
        }

        public String getMessageId() {
            return messageId;
        }

        public String getAccountId() {
            return accountId;
        }

        public int getAmountCents() {
            return amountCents;
        }
    }

    public static final class DedupStore {
        private final Set<String> seen = new HashSet<>();

        public boolean hasSeen(String messageId) {
            return seen.contains(messageId);
        }

        public void markSeen(String messageId) {
            seen.add(messageId);
        }
    }

    public static final class BalanceLedger {
        private final Map<String, Integer> balances = new HashMap<>();

        public void apply(String accountId, int amountCents) {
            balances.put(accountId, balances.getOrDefault(accountId, 0) + amountCents);
        }
```

```
        public int getBalance(String accountId) {
            return balances.getOrDefault(accountId, 0);
        }
    }

    public static final class PaymentConsumer {
        private final DedupStore dedupStore;
        private final BalanceLedger ledger;
        private int appliedCount = 0;
        private int duplicateCount = 0;

        public PaymentConsumer(DedupStore dedupStore, BalanceLedger ledger) {
            this.dedupStore = dedupStore;
            this.ledger = ledger;
        }

        public String handle(Message message) {
            if (dedupStore.hasSeen(message.getMessageId())) {
                duplicateCount += 1;
                return "skipped duplicate " + message.getMessageId();
            }

            ledger.apply(message.getAccountId(), message.getAmountCents());
            dedupStore.markSeen(message.getMessageId());
            appliedCount += 1;
            return "applied " + message.getMessageId() + " amount=" + message.getAmountCents();
        }

        public int balance(String accountId) {
            return ledger.getBalance(accountId);
        }

        public int getAppliedCount() {
            return appliedCount;
        }

        public int getDuplicateCount() {
            return duplicateCount;
        }
    }
}
```

idempotent-consumer-sample\java\Main.java

```
import java.util.List;

public class Main {
    public static void main(String[] args) {
        IdempotentConsumerSystem.DedupStore dedup = new IdempotentConsumerSystem.DedupStore();
        IdempotentConsumerSystem.BalanceLedger ledger = new IdempotentConsumerSystem.BalanceLedger();
        IdempotentConsumerSystem.PaymentConsumer consumer = new IdempotentConsumerSystem.PaymentConsumer(de

        List<IdempotentConsumerSystem.Message> messages = List.of(
            new IdempotentConsumerSystem.Message("msg-1", "acct-1", 1500),
            new IdempotentConsumerSystem.Message("msg-2", "acct-1", 2500),
            new IdempotentConsumerSystem.Message("msg-1", "acct-1", 1500),
            new IdempotentConsumerSystem.Message("msg-3", "acct-1", -700),
            new IdempotentConsumerSystem.Message("msg-2", "acct-1", 2500),
            new IdempotentConsumerSystem.Message("msg-4", "acct-2", 900)
        );

        for (IdempotentConsumerSystem.Message message : messages) {
            System.out.println(consumer.handle(message));
        }

        System.out.println("acct-1 balance: " + consumer.balance("acct-1"));
        System.out.println("acct-2 balance: " + consumer.balance("acct-2"));
        System.out.println("applied_count: " + consumer.getAppliedCount());
        System.out.println("duplicate_count: " + consumer.getDuplicateCount());
    }
}
```